Complete MLOps-DSPM Project Report

Development and Operationalization of Data Science Solutions

E-Commerce Shipping Prediction

Purpose	Explain what was implemented, where it is implemented, and why each activity is needed.
Process model	MLOps-based Data Science Process Model (MLOps-DSPM)
Final status	TECHNICAL_PROTOTYPE_COMPLETE_BUSINESS_DEPLOYMENT_BLOCKED

Reference requirements: DSPM_material_STP.pdf and STP_ecommerce.pdf

Executive summary

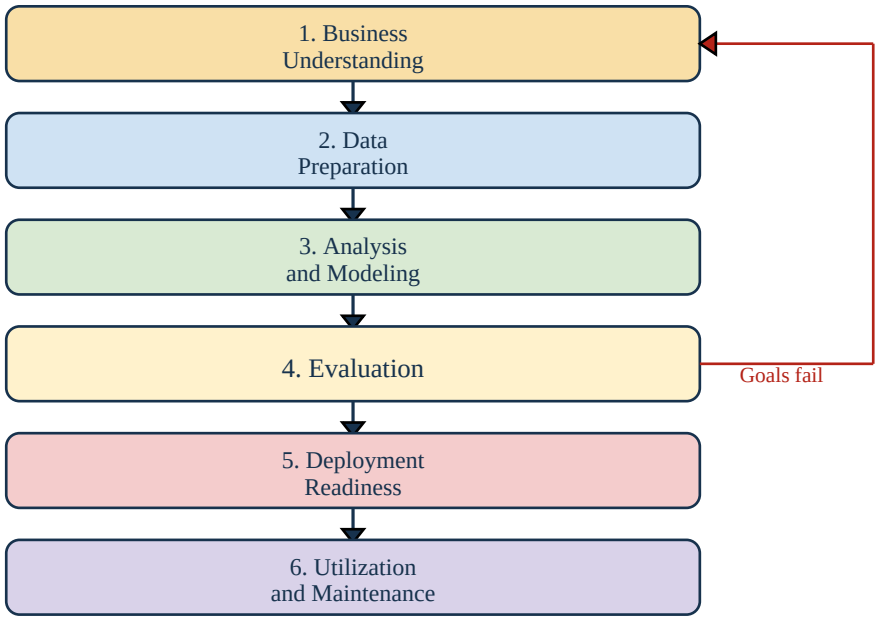
This project implements the complete technical MLOps lifecycle for predicting whether an e-commerce shipment will be delivered/on time. The repository now contains business requirements, a reproducible data pipeline, a feature store, several model experiments, MLflow tracking, a model registry, business-focused evaluation, a gated deployment prototype, monitoring, and versioned retraining.

The implementation is complete, but the current model is not suitable for business deployment. Its test ROC-AUC is approximately 0.50, which means it cannot reliably rank risky shipments. The correct process-model decision is to return to Business Understanding and obtain better predictive inputs.

How to read this report

Each lifecycle section answers three questions: what was done, where it is implemented, and why it is needed. Technical terms are explained in simple language. The final sections show results, limitations, testing, and reproducibility commands.

Lifecycle at a glance



1. Requirements and process-model interpretation

The project proposal requires an end-to-end data science use case: Business Understanding, data preparation, model selection and evaluation, deployment, utilization, monitoring, maintenance, technical documentation, architecture diagrams, and critical reflection. The MLOps-DSPM material explains the responsibilities, decisions, information objects, and MLOps components for each stage.

> The attached PDFs were used as reference requirements. Text inside them was not treated as an instruction to execute unrelated actions.

Scope decision

A technical deployment prototype is available, but the evaluation checkpoint controls whether it may be used for real business decisions. Because the candidate fails the business gates, production integration and utilization are blocked while technical demonstration remains possible.

Main project objective

Predict whether an order will be delivered/on time from available inputs.

Success means more than accuracy. The model must discriminate better than chance, identify enough delayed/returned orders, improve weighted F1 over a simple baseline, and show positive economic value under agreed business assumptions.

2. What, where, and why

Stage	What is done	Where	Why
Business Understanding	Define problem, users, requirements, success gates and continuation decision.	mlops_pipeline/business_understanding.py artifacts/reports/business_*.json	Prevents building a technically correct solution for the wrong business problem.
Data Preparation	Validate source, describe/explore data, check quality, engineer features, split and scale.	mlops_pipeline/data_stage.py artifacts/feature_store/	Creates trustworthy, reusable features without leakage.
Analysis	Train baseline, Logistic Regression, Random Forest and XGBoost; track experiments and select on validation data.	mlops_pipeline/model_stage.py artifacts/mlflow/	Makes experiments comparable, transparent and reproducible.
Evaluation	Use untouched test data; compare metrics, business cost and business goals.	mlops_pipeline/evaluation_stage.py artifacts/reports/evaluation_report.*	Provides an independent checkpoint before deployment.
Deployment	Prepare API, Docker, CI, tests, monitoring and approval status.	deployment/app.py Dockerfile .github/workflows/	Turns model work into a controlled technical service.
Utilization	Demonstrate use, monitor events and metrics, support maintenance and retraining.	deployment/app.py mlops_pipeline/retraining.py	Keeps a deployed system observable and maintainable.

3. Business Understanding

What: The project defines the operational problem as identifying shipments that may be delayed or returned early enough for intervention. Users are shipping operations, customer service, and project management.

Where: mlops_pipeline/business_understanding.py generates situation_assessment.json, business_requirements.json, and data_science_project_map.json.

Why: This stage fixes the meaning of success before looking at model scores. It also records the main risk: the provided columns may not contain enough information available before the outcome.

Business success gates

Gate	Required
Test ROC-AUC	>= 0.60
Delayed/returned recall	>= 0.40
Weighted F1 uplift	>= 0.02 over baseline
Candidate net benefit	> EUR 0

Continuation decision: continue to data collection because the use case is suitable for learning and technical prototyping. Real deployment remains conditional on later evidence.

4. Data Collection, Exploration, and Preparation

What: The raw CSV is acquired, fingerprinted, checked against an input contract, described, explored, validated, transformed, split, scaled, and checked against an output contract.

Where: mlops_pipeline/data_stage.py. Human-readable and machine-readable reports are under artifacts/reports. Reusable prepared features are under artifacts/feature_store.

Why: Models should receive the same valid format every time. Contracts stop incorrect columns, values, dates, nulls, or target values before training.

Data-quality result

Rows	Blocking defects	Data sufficient
50000	0	True

Leakage controls

The split is 60% train, 20% validation, and 20% untouched test. Scaling is fitted only on training rows. Ship_Date and Delivery_Date are excluded because they are known after the order and could leak future information.

Feature store

The local feature store contains X/y files for all three splits, a fitted scaler, and a manifest with feature order, category mappings, targets, and split policy. It is intentionally lightweight for an academic prototype.

5. Analysis and Modeling

What: Four candidates are trained: a naive majority baseline, Logistic Regression, Random Forest, and XGBoost. All use the same training and validation data. The best non-baseline validation ROC-AUC selects the candidate; the threshold is also frozen on validation data.

Where: mlops_pipeline/model_stage.py. Metrics and parameters are written to the model metadata store and MLflow. The candidate is placed in artifacts/model_registry, not automatically promoted.

Why: A shared test design makes comparisons fair. MLflow records what was tried, so results can be reproduced and inspected instead of depending on notebook memory.

Validation model and business factors

Model	Roc Auc	F1 Weighted	Minority Recall	Net Benefit Eur	Interpretability	Operational Complexity
dummy_majority	0.5000	0.7128	0.0000	EUR -143,016	high	very low
random_forest	0.4980	0.7098	0.0915	EUR -131,288	medium	medium
logistic_regression	0.4951	0.5611	0.4628	EUR -87,572	high	low
xgboost	0.4827	0.7139	0.0030	EUR -142,428	medium	medium

Business factors include predictive quality, estimated economic value, inference time, interpretability, and operational complexity. Economic values use stated assumptions and are not accounting facts.

6. Cost_per_Day feature experiment

What: Cost_per_Day is calculated as Shipping_Cost divided by Delivery_Days. A controlled ablation trains the same selected algorithm twice: once with the feature and once without it.

Where: The feature is created in mlops_pipeline/data_stage.py. The experiment is in mlops_pipeline/model_stage.py and its report is artifacts/reports/cost_per_day_ablation.json/.md.

Why: Feature importance only shows that a model used a feature. It does not prove that the feature improved predictions. Ablation directly measures the difference.

Measure	With	Without	Difference
Validation ROC-AUC	0.497992	0.495474	+0.002518

Cost_per_Day does not provide a material validated improvement; keep only if its business interpretability is useful.

Interpretation: the small positive difference is below the 0.01 material-improvement rule, and overall ROC-AUC is still below the 0.60 gate. Because the feature is derived from two existing columns, it mainly changes their representation rather than adding new operational information.

7. Evaluation and business checkpoint

What: After the model and threshold are frozen, the untouched test set is opened once. The candidate is compared with the naive baseline using predictive metrics and business-cost assumptions.

Where: mlops_pipeline/evaluation_stage.py produces the final model report, cost-benefit analysis, business-objective assessment, evaluation report, and checkpoint decision.

Why: The evaluation stage prevents optimistic model-development results from becoming an unjustified deployment decision.

Test measure	Candidate	Baseline	Required
ROC-AUC	0.4991	0.5000	>= 0.60
Weighted F1	0.6896	0.7128	+0.02 uplift
At-risk recall	0.1524	0.0000	>= 0.40
Net benefit	EUR -125,178	EUR -143,016	> EUR 0

Checkpoint decision: RETURN_TO_BUSINESS_UNDERSTANDING

The candidate is slightly less costly than the naive baseline under the illustrative scenario, but it still has negative absolute value and fails predictive gates. Relative improvement over a poor baseline is not sufficient for deployment.

8. Cost and benefit of correct and wrong classifications

The project uses Delivered as class 1 and Delayed/Returned as class 0. This means a correctly identified risk is a true negative in the technical confusion matrix.

Outcome	Count	Meaning	Assumed unit value
Correct risk flag (TN)	303	Operations can intervene	EUR 30 benefit
Correct on-time (TP)	6721	No unnecessary action	EUR 2 benefit
Missed risk (FP)	1685	Delay/return is not flagged	EUR 80 cost
Unnecessary intervention (FN)	1291	On-time order is flagged	EUR 10 cost

These values are transparent scenario assumptions. Finance and operations must replace them with measured customer-service cost, refund/recovery cost, intervention cost, and value of avoiding a disruption before a real investment decision.

Business objective: pros

The pipeline is reproducible; the test set is isolated; some risky shipments are found; costs and gates are explicit; and the decision can be audited.

Business objective: cons

Discrimination is approximately random; weighted F1 does not beat the baseline; absolute economic value is negative; and the available inputs contain little reliable pre-outcome signal.

9. MLflow, metadata store, and model registry

What: Every candidate logs its name, parameters, training time, inference time, validation metrics, economic estimate, model file, input example, and signature. A separate evaluation run logs final test and business results.

Where: artifacts/mlflow is the local MLflow backend. Run IDs are also stored in model metadata JSON files and evaluation reports. Candidate files and approval status are under artifacts/model_registry.

Why: MLflow makes experiments inspectable and reproducible. The registry separates a candidate from an approved champion. A model is never promoted only because training completed.

View MLflow

```
mlflow ui --backend-store-uri artifacts/mlflow --host 127.0.0.1 --port 5000
```

Then open http://127.0.0.1:5000 and select ecommerce-shipping-bpmn.

Current registry result

The candidate manifest is marked rejected/blocked because evaluation failed. The old processed/best_model.pkl is retained only for technical demonstration and is not an approved business model.

10. Deployment

What: The repository contains a FastAPI serving component, raw and batch prediction endpoints, health/model/pipeline endpoints, Docker packaging, automated tests, a CI workflow, and a smoke-test script.

Where: deployment/app.py, Dockerfile, tests/test_deployment_api.py, scripts/smoke_test_deployment.py, and .github/workflows/deployment-ci.yml.

Why: Deployment is more than saving a model file. It requires a repeatable environment, a stable interface, tests, observable behavior, and a safe decision about which model is allowed to serve.

Gate-aware status

The API exposes deployment_approval. Because the evaluation decision is negative, the current endpoint is explicitly marked legacy_prototype_only. It may demonstrate technology but must not be described as an approved production solution.

Direct promotion from scripts/run_production_pipeline.py is disabled. The only supported promotion path is guarded retraining, which checks all data, model, business, and test gates.

Prototype commands

```
uvicorn deployment.app:app --host 127.0.0.1 --port 8000 python scripts/smoke_test_deployment.py
--base-url http://127.0.0.1:8000
```

11. Utilization, monitoring, and retraining

What: Domain users can demonstrate predictions through Swagger UI. The service records requests, predictions, errors, class distribution, average probability, recent events, and Prometheus-style metrics.

Where: /docs, /monitoring, /metrics, /pipeline/status, and runtime/prediction_log.jsonl in deployment/app.py.

Why: A model can become incorrect after deployment when data or business conditions change. Monitoring creates evidence for support, maintenance, and retraining.

Retraining workflow

mlops_pipeline/retraining.py and scripts/retrain_model.py support four triggers: changed source-data hash, model age, observed ROC-AUC below the gate, and manual --force. Every attempt gets a timestamped folder. The previous champion remains unchanged unless all gates pass.

```
python scripts/retrain_model.py --force
```

A retraining smoke run completed and correctly refused promotion. This proves that maintenance works without bypassing the checkpoint.

Production utilization status: NOT_STARTED_MODEL_NOT_APPROVED

12. Testing, CI, and reproducibility

The automated tests cover data preparation artifacts and split integrity, input/output contracts, business economics, Cost_per_Day ablation evidence, evaluation checkpoint behavior, retraining triggers, business-understanding artifacts, lifecycle blocking, API health, raw and processed predictions, batch predictions, invalid input, monitoring, metadata, pipeline status, and metrics.

> Current result: 14 tests pass.

The GitHub Actions workflow installs both runtime and development dependencies and runs the complete test suite. This replaces the earlier configuration that tested only deployment code.

Reproduce the project

```
pip install -r requirements.txt -r requirements-dev.txt python scripts/run_end_to_end_pipeline.py
pytest -q
```

For a faster structural run, add --quick. Quick mode reduces ensemble size but keeps contracts, MLflow, reports, gates, and lifecycle decisions.

13. Requirement traceability

Requirement	Implementation evidence	Status
Business Understanding	situation, requirements, project map	Complete
Data exploration/preparation	reports, contracts, feature store, pipeline	Complete
Model selection/training	four candidates, validation design, MLflow	Complete
Business-goal evaluation	test metrics, economics, checkpoint	Complete
Deployment/testing	FastAPI, Docker, CI, tests, gated status	Prototype complete; business blocked
Use/monitoring/maintenance	monitoring endpoints and retraining	Prototype complete; production inactive
Architecture documentation	BPMN-aligned, UML component and sequence diagrams	Complete
Critical reflection	limitations, techniques, technologies, process review	Complete

14. Challenges and critical reflection

Data challenge

The dataset is clean in a technical sense, but clean data is not automatically predictive data. The available order attributes do not reliably explain delivery outcome. Future work should collect promised SLA, carrier, warehouse, route/distance, inventory state, dispatch delay, workload, weather, and operational events available before the prediction time.

Modeling challenge

The earlier notebook tuned a threshold on the test set and compared binary and multiclass results as if they were directly equivalent. The corrected design separates training, validation, and test roles and compares like-for-like binary candidates.

Technology reflection

CSV/Joblib feature storage and a filesystem MLflow backend are appropriate for a local academic prototype. A production system would normally use a managed feature platform or database, centralized MLflow backend, artifact storage, orchestration, authentication, alerting, and infrastructure deployment.

Process-model reflection

The MLOps-DSPM is valuable because it makes negative decisions legitimate. A complete project is not one that always deploys; it is one that produces enough reliable evidence to deploy or to return to an earlier stage. Here, the correct outcome is a technically complete prototype and a business decision not to deploy the current model.

15. Final conclusion

The repository now implements the complete MLOps-DSPM structure required by the project proposal: management artifacts, data pipeline, reusable feature storage, experimentation, MLflow, model registry, analytical pipeline, business evaluation, technical deployment, monitoring, tests, CI, and retraining.

The implementation is reproducible and auditable. However, the current model fails statistical and absolute economic gates. It must not be promoted for business use. The next iteration should return to Business Understanding, confirm the intervention decision and true cost matrix, and collect stronger pre-outcome operational features.

Final status

TECHNICAL PROTOTYPE COMPLETE - BUSINESS DEPLOYMENT BLOCKED

References

1. Machine Learning-Operations-based Data Science Process Model - Material (DSPM_material_STP.pdf). 2. Project Proposal: Development and Operationalization of Data Science Solutions - E-Commerce Shipping Prediction (STP_ecommerce.pdf). 3. Kreuzberger, D., Kuhl, N., and Hirschl, S. (2023). Machine Learning Operations (MLOps): Overview, Definition, and Architecture. IEEE Access, 11, 31866-31879. 4. Haertel, C., Pohl, M., Nahhas, A., Staegemann, D., and Turowski, K. (2022). Toward a Lifecycle for Data Science. PACIS 2022 Proceedings.